Project Title	**Real-Time Face Segmentation for Movie Cast Identification**
Skills take away From This Project	**Python, Computer Vision, OpenCV, Streamlit**
Domain	**Entertainment**

Problem Statement:

Scene Cast AI: Real-Time Face Segmentation for Movie Cast Identification. Company X's streaming app needs to automatically detect and segment faces in movie scene screenshots so users can pause videos and instantly view cast/crew details for actors on screen.

Business Domain Value

- Pause-and-identify feature: Users pause movies to see actor names/profiles overlayed on detected faces.
- Personalized recommendations: Track viewer preferences for specific actors across films.
- Content moderation: Automatically flag inappropriate scenes via face detection.
- Advertising: Dynamic ads featuring favourite actors during streaming breaks.

Approach:

1. **Pre-Processing, Data Visualization, EDA:**
    - Load dataset of movie scene images and corresponding binary face masks.
    - Preprocess: Resize to 256x256, augment (flip/rotate) for varied poses/lighting.

2. **Model Building:**
    - Build U-Net model with MobileNetV2 encoder (transfer learning), custom decoder with skip connections.
    - Train the mode
    - Implement custom Dice Coefficient metric and Dice Loss function.
    - To deal with large training time, save the weights so that you can use them when training the model for the second time without starting from scratch.

3. **Test the Model, Fine-tuning and Repeat:**
    - Test the model and report as per evaluation metrics
    - Try different models
    - Set different hyper parameters, by trying different optimizers, loss functions, epochs, learning rate, batch size, checkpointing, early stopping etc. for these models to fine-tune them
    - Report evaluation metrics for these models along with your observation on how changing different hyper parameters leads to change in the final evaluation metric

4. **Deployment:**
    - Integrating the trained model into a Streamlit-based web application.
    - Providing an interface where users can upload images or use real-time webcam feeds for detection.

Streamlit Integration:

To ensure user-friendly access to the model, a Streamlit-based web application is developed with the following features:

- **Image Upload:**
  Users can upload images.

- **Download & Export:**

Users can download detection logs for future reference.

- **Performance Dashboard:**
  Displays key metrics such as processing time, detection accuracy, and confidence scores.

Deployment (Completely Optional):

Try to deploy it on any platforms like Hugging Face, AWS, Azure etc.

Project Evaluation Metrics:

Below matrix should be used for the evaluation

- Dice Coefficient (>0.92): Measures mask overlap precision.
- IoU (>0.88): Intersection over Union for segmentation quality.
- Inference Speed (<100ms/image): Real-time app performance.
- F1-Score (>0.90): Balances precision/recall for face detection.

While we encourage peer collaboration and contribution, plagiarism, copying the code from other sources or peers will defeat the purpose of coming to this program. We expect the highest order of ethical behaviour.

Data: https://drive.google.com/drive/folders/1dBLCTJUU8DR8sHbjCESPafETLVWJ-z5S?usp=sharing

Results:

Learners should achieve Dice Coefficient >0.92 on validation set, with real-time inference (<100ms/image) and accurate face masks even in crowded/complex movie scenes. Model ready for Streamlit app integration showing overlaid bounding boxes

Technical Tags:

- Python
- OpenCV
- Streamlit
- **Deep Learning:** YOLO object detection model.
- **Computer Vision:** OpenCV for preprocessing and visualization.
- **OCR Technology:** Tesseract/EasyOCR for text recognition.
- **Web Application:** Streamlit for interactive deployment.
- **Python Libraries:** NumPy, Pandas, Matplotlib, Seaborn, TensorFlow/PyTorch.
- **Image Processing:** Includes resizing, contrast enhancement, and edge detection techniques.

Project Deliverables:

- Jupiter notebook with EDA, model training, evaluation.
- Trained U-Net model weights (.h5).
- Custom Dice loss/metric functions.
- Streamlit demo app with image upload and mask visualization.
- Performance report (metrics table, sample predictions).
- GitHub repository with README and requirements.txt.
- 2-minute demo video.

Optional Project

LLM

Objective / Problem Statement

With the growing need for efficient document search and question-answering, traditional search engines often return unrelated or incomplete results. The objective of this project is to build a **RAG (Retrieval-Augmented Generation) system** using the **Llama 2** model, where users can ask questions, and the model retrieves relevant information from uploaded **PDFs and documents** before generating an answer. This ensures more **accurate, context-aware responses**than standalone LLMs.

Approach

Step 1: Data Ingestion

- Accept **PDF, DOCX, or TXT** documents as input.
- Extract text using **PyMuPDF (fitz), PDFPlumber, or docx2txt**.
- Store extracted text in an **embedding database (FAISS, ChromaDB, or Pinecone).**

Step 2: Text Embeddings & Indexing

- Convert document text into **vector embeddings** using **SentenceTransformers** or **Hugging Face's embedding models**.
- Store vectors in a **vector database** for efficient retrieval.

Step 3: Query Processing & Retrieval

- When a user asks a question, convert it into an embedding.
- Use a **similarity search algorithm** (e.g., cosine similarity) to retrieve the most relevant text chunks.

Step 4: Response Generation

- Pass retrieved context and query to **Llama 2** for response generation.
- Fine-tune or prompt-engineer the Llama 2 model for improved answers.

Step 5: Evaluation & Optimization

- Compare responses with ground truth answers.
- Use **BLEU, ROUGE, or embedding-based similarity metrics** to evaluate searches.
- More **context-aware responses** compared to standard LLM outputs.
- A scalable solution that can be extended to multiple documents and domains.

Technologies & Tools

accuracy.

Expected Outcome

- A **Q&A chatbot** that accurately answers user queries from uploaded documents.
- Improved **retrieval accuracy** using embeddings instead of keyword-based
- **LLM Model:** Llama 2 (Meta)
- **Embedding Models:** SentenceTransformers, OpenAI Embeddings
- **Vector Database:** FAISS, ChromaDB, Pinecone
- **Document Processing:** PyMuPDF, PDFPlumber, docx2txt
- **Frameworks:** LangChain, LlamaIndex, Hugging Face
- **Frontend Must:** Streamlit

Timeline

Analyse data EDA Ploting Building Model ML Model Selection	6 Days
Deep Learning	5 Days
Streamlit	3 Days
Total	2 weeks

The project must be completed and submitted **within 10 days from the assigned Date.**

GUVI | HCL

Skill Up. Level Up

PROJECT DOUBT CLARIFICATION SESSION (PROJECT AND CLASS DOUBTS)

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively. **Note: Book the slot at least before 12:00 Pm on the same day**

Timing: Tuesday, Thursday, Saturday (5:00PM to 7:00PM)

Booking link :https://forms.gle/XC553oSbMJ2Gcfug9

LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation. **Note: This form will Open on Saturday and Sunday Only on Every Week**

Timing: Monday-Saturday (11:30PM to 12:30PM)

Booking link : https://forms.gle/1m2Gsro41fLtZurRA